



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO1 ASSESSMENT 2

AGILE SPRINT PLANNING & USER STORY WORKSHOP

INTELLIGENT AI-BASED SMART PUBLIC TRANSPORTATION DEMAND FORECASTING SYSTEM



NAME: N.RAMYA

REG.No: 192511403

COUESE CODE: CSA1017

COURSE NAME: SOFTWARE ENGINEERING

FACULTY: LATHA.R

Table of Contents

- Project Background & Problem Statement
- Project Vision and Stakeholders
- Epics and User Stories
- Acceptance Criteria
- Product Backlog Prioritization
- Sprint Backlog Planning (Sprint 1)
- Story Point Estimation
- Sprint Goal and Expected Deliverables

INTRODUCTION

Public transportation is a cornerstone of sustainable urban mobility, yet most transit agencies still rely on static, historically-averaged timetables that cannot react to real-world fluctuations in demand. This report presents a systematic case study analysis of Problem Statement 20: an Intelligent AI-Based Smart Public Transportation Demand Forecasting System. The report begins by unpacking the problem, its objectives, and its scope, before identifying the stakeholders affected and the challenges they currently face. It then critically examines the limitations of today's manual, reactive scheduling approach and proposes a comprehensive, AI-driven software solution built on sound software engineering principles — modularity, scalability, maintainability, reliability, usability, and security. An Agile Scrum development methodology is justified as the most suitable approach for delivering and maintaining the platform, supported by a sample sprint roadmap. The report closes with an evaluation of expected outcomes, potential risks and limitations, and directions for future enhancement, giving a complete, implementation-oriented view of how AI, IoT, GPS, and Big Data Analytics can be combined to make public transportation more efficient, reliable, and commuter-friendly.

Recent advancements in **Artificial Intelligence (AI)**, **Machine Learning (ML)**, **Internet of Things (IoT)**, **Global Positioning System (GPS)**, and **Big Data Analytics** have created new opportunities to improve public transportation services. By analyzing historical ridership data along with real-time information such as weather conditions, traffic congestion, special events, and GPS location data, AI can accurately forecast passenger demand and assist transit authorities in making informed decisions. This enables better scheduling of buses and trains, optimized fleet allocation, and timely communication with passengers.

The purpose of this report is to analyze the given problem statement, identify the key stakeholders and challenges, examine the limitations of the existing system, propose a suitable software solution, justify the selection of an appropriate Software Development Life Cycle (SDLC) methodology, and discuss the expected outcomes, risks, limitations, and future enhancements. Through this analysis, the report demonstrates how software engineering concepts can be applied to solve real-world transportation problems and contribute to the development of smarter, more sustainable public transportation systems.

1. Project Background & Problem Statement

Industry Context

Public transportation agencies strive to provide reliable and efficient services while managing fluctuating passenger demand. AI, IoT, GPS, and Big Data Analytics enable transit operators to predict passenger demand, optimize schedules, and improve resource allocation.

Problem Statement

Public transportation systems often face overcrowding during peak hours and underutilization during off-peak periods due to inaccurate demand estimation. Traditional scheduling approaches lack predictive capabilities and real-time adaptability. An AI-powered transportation demand forecasting platform should analyze historical ridership data, weather conditions, events, and traffic patterns to predict passenger demand. The system should recommend optimized bus or train schedules, improve fleet allocation, and provide real-time passenger information to enhance service quality.

Objectives

- Forecast passenger demand using AI.
- Optimize public transport schedules.
- Improve fleet allocation.
- Monitor passenger flow in real time.
- Analyze travel patterns.
- Generate transportation analytics.
- Reduce overcrowding.
- Improve public transport efficiency.

Expected Outcomes

- Improved service reliability.
- Reduced passenger waiting time.
- Better fleet utilization.
- Increased commuter satisfaction.
- Lower operational costs.

- Accurate demand forecasting.
- Enhanced urban mobility.
- Sustainable public transportation management.

1. PROJECT VISION AND STAKEHOLDERS

1.1 Project Vision

To build an intelligent, data-driven public transportation platform that leverages AI, IoT, GPS, and Big Data Analytics to accurately forecast passenger demand, optimize bus and train schedules, and dynamically allocate fleet resources — reducing overcrowding, minimizing wait times, and enhancing the overall commuter experience while lowering operational costs for transit agencies.

1.2 Project Objectives (Refined)

- Deliver accurate, near real-time passenger demand forecasts using historical ridership, weather, event, and traffic data.
- Recommend optimized schedules and fleet deployment that align capacity with predicted demand.
- Provide passengers with real-time, reliable transit information to improve trip planning.
- Equip transit planners and executives with analytics dashboards for data-driven decision-making.
- Ensure the platform is secure, scalable, and integrates cleanly with existing IoT/GPS fleet infrastructure.

1.3 Key Stakeholders

Stakeholders were identified by mapping every actor who provides input to, is impacted by, or makes decisions about the system — spanning business sponsors, operational users, the Scrum team, and external data/regulatory parties. Each is classified below by category and mapped to their primary interest in the project.

Stakeholder	Category	Interest / Role
Public Transportation Agency (Sponsor)	Business Owner	Funds and owns the platform; accountable for service quality and ROI.
Passengers / Commuters	End User	Primary beneficiaries; need reliable, real-time transit information.

Bus / Train Drivers & Operations Staff	Operational User	Execute optimized schedules generated by the system.
Fleet Managers	Operational User	Approve and act on fleet allocation and scheduling recommendations.
Maintenance Coordinators	Operational User	Ensure vehicle availability data feeds into allocation decisions.
Data Scientists / ML Engineers	Development Team	Design, train, and maintain the demand-forecasting models.
Software Developers (Scrum Team)	Development Team	Build and integrate the platform's front-end, back-end, and APIs.
IT Infrastructure / DevOps Team	Development Team	Manage IoT/GPS data pipelines, cloud infrastructure, and security.
Scrum Master	Agile Facilitator	Facilitates ceremonies, removes impediments, coaches the team.
Product Owner	Agile Facilitator	Owns and prioritizes the product backlog based on stakeholder value.
City Traffic / Urban Planning Dept.	External Stakeholder	Supplies traffic data; uses analytics for infrastructure planning.
Weather Data Provider (3rd Party)	External Stakeholder	Supplies weather feeds consumed by the forecasting engine.
Regulatory / Government Bodies	External Stakeholder	Ensure compliance with data privacy and transportation regulations.

1.4 RACI Matrix (Key Activities)

To avoid ambiguity about decision rights during sprint execution, the table below defines who is Responsible, Accountable, Consulted, and Informed for the major recurring activities.

Activity	Product Owner	Scrum Master	Dev Team	Fleet Mgr.	Data Scientist
Backlog prioritization	A/R	C	I	C	I
Sprint planning	R	A	R	I	R
Model training & validation	I	I	C	I	A/R
Schedule approval	I	I	I	A/R	I
Sprint review / demo	A	R	R	C	C
Retrospective	C	A/R	R	I	I

R = Responsible (does the work), A = Accountable (owns the outcome), C = Consulted (gives input), I = Informed (kept up to date).

2. EPICS AND USER STORIES

An Epic is a large body of work that is too big to be completed in a single sprint and is broken down into smaller, independently deliverable user stories. Decomposing the problem statement into epics keeps the backlog organized around end-to-end value streams (forecasting → scheduling → fleet allocation → passenger experience → analytics → platform infrastructure), which makes it easier to sequence sprints, assign ownership, and track progress at both a strategic and a tactical level.

The product requirements have been decomposed into six Epics, each broken down into detailed user stories following the standard Agile format: “As a <user>, I want <feature>, so that <benefit>.” Each story is written to be independent, negotiable, valuable, estimable, small, and testable (the INVEST principle).

2.1 Epic Summary

Epic	Description	# Stories	Total Pts
Epic A	AI Demand Forecasting Engine — the predictive core of the platform	5	31
Epic B	Schedule Optimization — turns forecasts into actionable timetables	3	16
Epic C	Fleet Allocation Management — deploys vehicles where demand is highest	3	18
Epic D	Real-Time Passenger Information — the commuter-facing experience	3	16
Epic E	Travel Pattern & Analytics Dashboard — strategic insight for executives	3	13
Epic F	System Administration & Data Integration — platform plumbing & security	3	16
	Total Product Backlog	20	110

At the team's estimated velocity of ~37–38 points per sprint, the full backlog above represents roughly three sprints of work — giving the Product Owner a realistic basis for release planning beyond Sprint 1.

Epic A — AI Demand Forecasting Engine

- US-01 As a transit planner, I want the system to analyze historical ridership data, so that I can understand and predict passenger demand patterns.
- US-02 As a transit planner, I want the system to incorporate weather data into demand predictions, so that forecasts account for weather-related ridership changes.
- US-03 As a transit planner, I want the system to factor local events (concerts, sports, holidays) into demand forecasts, so that schedules can be adjusted for event-driven surges.
- US-04 As a transit planner, I want the system to analyze traffic pattern data, so that demand forecasts reflect road congestion impacts on travel time.
- US-05 As a data scientist, I want to retrain the forecasting ML model periodically with new data, so that prediction accuracy improves continuously.

Epic B — Schedule Optimization

- US-06 As a fleet manager, I want the system to recommend optimized bus/train schedules based on forecasted demand, so that overcrowding and underutilization are reduced.
- US-07 As a fleet manager, I want to review and approve AI-recommended schedule changes before they go live, so that I retain operational oversight.
- US-08 As an operations manager, I want to be alerted when demand is expected to exceed current capacity, so that I can proactively add extra trips.

Epic C — Fleet Allocation Management

- US-09 As a fleet manager, I want the system to recommend optimal fleet allocation across routes, so that vehicles are deployed where demand is highest.
- US-10 As a fleet manager, I want a real-time dashboard of fleet location and status, so that I can monitor ongoing operations.
- US-11 As a maintenance coordinator, I want vehicle availability and maintenance schedules factored into allocation recommendations, so that allocation plans stay realistic.

Epic D — Real-Time Passenger Information

- US-12 As a commuter, I want to view real-time bus/train arrival predictions on a mobile app, so that I can plan my trip and minimize waiting time.

- US-13 As a commuter, I want to receive notifications about delays or schedule changes, so that I can adjust my travel plans accordingly.
- US-14 As a commuter, I want to see crowding-level indicators for upcoming vehicles, so that I can choose less crowded options when possible.

Epic E — Travel Pattern & Analytics Dashboard

- US-15 As a transit executive, I want a dashboard of key ridership analytics (peak times, popular routes), so that I can make informed strategic decisions.
- US-16 As a transit executive, I want to generate exportable performance reports, so that I can share insights with city officials.
- US-17 As an urban planner, I want to analyze long-term travel pattern trends, so that I can plan future infrastructure investments.

Epic F — System Administration & Data Integration

- US-18 As a system administrator, I want to integrate IoT/GPS data feeds from vehicles in real time, so that the platform has accurate live location data.
- US-19 As a system administrator, I want role-based access control for different user types, so that data security and privacy are maintained.
- US-20 As a system administrator, I want to configure system alerts and thresholds, so that the platform can be tailored to agency-specific needs.

3. ACCEPTANCE CRITERIA

Acceptance criteria translate a user story into specific, testable conditions that must be true for the story to be considered complete. They remove ambiguity for developers, give QA a concrete basis for test cases, and give the Product Owner an objective way to accept or reject the work during Sprint Review. Each story below has at least two criteria, written in the Given–When–Then (Gherkin) format.

Clause	Meaning
Given	The precondition / context that must exist before the behaviour is triggered.
When	The specific action or event performed by the user or system.
Then	The expected, observable outcome that confirms the story works as intended.

Acceptance criteria for the six user stories selected for Sprint 1 (see Section 5) are defined below:

US-01: Analyze Historical Ridership Data

- Given at least 12 months of historical ridership data is available, When the forecasting engine runs, Then it must generate a demand prediction for each route with $\geq 85\%$ accuracy against a validation dataset.
- Given incomplete or missing data for a period, When the model processes that period, Then it must flag the gap and apply an approved data-imputation method rather than fail silently.

US-02: Incorporate Weather Data

- Given a live weather API feed is connected, When a forecast is generated, Then weather conditions (rain, temperature, snow) for the next 24-48 hours must be included as model input.
- Given severe weather is forecast, When the demand model runs, Then predicted ridership must adjust automatically and the adjustment must be visible in the forecast output.

US-06: Recommend Optimized Schedules

- Given a demand forecast for a route, When the scheduling engine runs, Then it must output a recommended timetable that keeps predicted vehicle occupancy within an 85% capacity threshold.
- Given the current published schedule, When a new recommendation is generated, Then the system must display a side-by-side comparison highlighting the changes.

US-07: Approve Schedule Changes

- Given an AI-generated schedule recommendation, When a fleet manager opens the review screen, Then they must be able to Approve, Reject, or Edit each proposed change.
- Given a schedule change is approved, When approval is submitted, Then the new schedule must be published to the live system within 5 minutes.

US-12: Real-Time Arrival Predictions

- Given a vehicle is in transit with an active GPS signal, When a commuter opens the app, Then the predicted arrival time for their selected stop must be displayed and refresh at least every 30 seconds.
- Given GPS signal is lost for a vehicle, When arrival time is requested, Then the app must display “Estimate Unavailable” rather than a stale or incorrect time.

US-13: Delay & Schedule Change Notifications

- Given a delay of more than 5 minutes is detected on a route, When the delay is confirmed, Then all subscribed commuters on that route must receive a push notification within 2 minutes.
- Given a commuter has opted out of notifications, When a delay occurs, Then no notification must be sent to that user.

4. PRODUCT BACKLOG PRIORITIZATION

The product backlog was prioritized using the MoSCoW technique (Must have, Should have, Could have, Won't have this time) combined with a Business Value score (1–10). Business value reflects stakeholder impact and alignment with the project vision, while priority additionally accounts for technical dependencies — for example, schedule optimization (Epic B) and passenger notifications (Epic D) cannot be reliably tested until the forecasting engine (Epic A) produces real predictions, so Epic A stories were pulled forward even where an individual story's business value was comparable to a lower-epic story.

4.1 MoSCoW Distribution

Priority	# Stories	Total Points	% of Backlog
Must Have	5	34	31%
Should Have	10	54	49%
Could Have	5	22	20%

4.2 Key Dependencies

Story	Depends On	Reason
US-06	US-01	Schedule optimization requires a working demand forecast to generate recommendations.
US-08	US-01, US-06	Capacity alerts compare forecasted demand against the optimized schedule.
US-09	US-01	Fleet allocation recommendations are driven by the same demand forecast output.
US-12	US-18	Real-time arrival predictions require the live GPS/IoT data feed to be integrated first.

US-13	US-12	Delay notifications are triggered from the same real-time position/arrival data.
US-15	US-01, US-10	Analytics dashboard aggregates forecast and fleet-status data from earlier epics.

Note: US-12 (real-time arrival predictions) was still included in Sprint 1 because the team will build against a simulated/mocked GPS feed until US-18 (full IoT integration, planned for Sprint 2) is complete — a common Agile technique for de-risking dependent stories.

	User Story (Summary)	Priority	Bus. Value	Story Pts
US-01	Analyze historical ridership data	Must	10	8
US-06	Recommend optimized schedules	Must	10	8
US-12	Real-time arrival predictions	Must	9	8
US-02	Incorporate weather data	Must	8	5
US-13	Delay / schedule notifications	Must	8	5
US-08	Capacity-exceeded alerts	Should	7	5
US-07	Approve schedule changes	Should	6	3
US-09	Optimal fleet allocation	Should	7	8
US-03	Event-driven demand factoring	Should	6	5
US-15	Ridership analytics dashboard	Should	6	5
US-04	Traffic pattern analysis	Should	5	5
US-05	Periodic model retraining	Should	6	8
US-10	Real-time fleet dashboard	Should	5	5
US-17	Long-term travel trend analysis	Should	5	5

US-19	Role-based access control	Should	4	5
US-11	Maintenance-aware allocation	Could	4	5
US-14	Crowding-level indicators	Could	4	3
US-16	Exportable performance reports	Could	3	3
US-18	IoT/GPS live data integration	Could	4	8
US-20	Configurable alert thresholds	Could	3	3

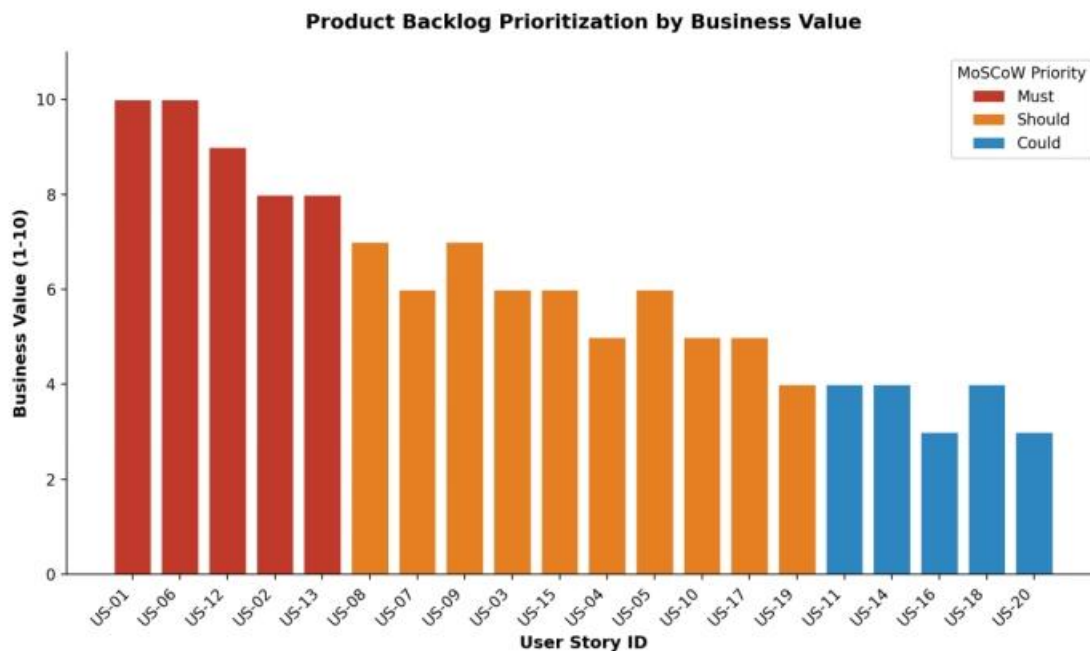


Figure 1: Product backlog prioritized by MoSCoW category and business value.

5. SPRINT BACKLOG PLANNING (SPRINT 1)

The six highest-priority “Must Have” stories were selected for Sprint 1, together forming a coherent, demoable MVP slice: forecast demand → optimize the schedule → inform passengers in real time.

ID	Selected User Story	Points
US-01	Analyze historical ridership data to understand demand patterns	8
US-02	Incorporate weather data into demand predictions	5
US-06	Recommend optimized bus/train schedules based on forecasted demand	8
US-07	Review and approve AI-recommended schedule changes	3
US-12	View real-time arrival predictions on a mobile app	8
US-13	Receive notifications about delays or schedule changes	5
	Total Sprint 1 Commitment	37

5.1 Sprint Ceremonies Schedule

Ceremony	When	Purpose
Sprint Planning	Day 1 (4 hrs)	Select backlog items, define the sprint goal, and break stories into tasks.
Daily Stand-up	Every day (15 min)	Sync on progress, plans for the day, and blockers.
Backlog Refinement	Day 5 (1 hr)	Groom and re-estimate upcoming backlog items for Sprint 2.
Sprint Review / Demo	Day 10 (2 hrs)	Demonstrate completed increments to the Product Owner and stakeholders.
Sprint Retrospective	Day 10 (1 hr)	Reflect on what went well/poorly and agree on process improvements.

5.2 Task Breakdown

US-01 — Analyze Historical Ridership Data (8 pts)

- Set up data ingestion pipeline from ridership database.
- Clean and pre-process historical data (handle missing values, outliers).
- Perform feature engineering (time-of-day, day-of-week, seasonality).
- Train and validate baseline forecasting model.
- Write unit tests for the data pipeline.

US-02 — Incorporate Weather Data (5 pts)

- Integrate third-party weather API.
- Build weather-feature transformation layer.
- Re-train forecasting model with weather features and validate accuracy uplift.

US-06 — Recommend Optimized Schedules (8 pts)

- Design schedule-optimization algorithm using forecast output.
- Build capacity-threshold logic (85% occupancy cap).
- Create schedule comparison view (current vs. recommended).
- Integration testing with US-01 forecast output.

US-07 — Approve Schedule Changes (3 pts)

- Build fleet-manager review UI (Approve / Reject / Edit).
- Implement publish-to-live workflow with 5-minute SLA.

US-12 — Real-Time Arrival Predictions (8 pts)

- Integrate live GPS feed into mobile app back-end.
- Build arrival-time prediction API (30-second refresh).
- Handle GPS-signal-lost fallback state (“Estimate Unavailable”).
- Develop mobile app UI for arrival display.

US-13 — Delay & Schedule Notifications (5 pts)

- Build delay-detection service (> 5-minute threshold).
- Integrate push-notification service.
- Implement user notification-preference / opt-out settings.

6. STORY POINT ESTIMATION

Story points were assigned using Planning Poker with the Fibonacci sequence (1, 2, 3, 5, 8, 13...), reflecting relative complexity, effort, and uncertainty rather than absolute time. The non-linear gaps between numbers force the team to make a clear judgment call rather than hair-splitting between, say, an 8 and a 9.

6.1 Planning Poker Process

- The Product Owner reads the story aloud and answers any clarifying questions from the team.
- Each team member privately selects a Fibonacci card representing their estimate.
- All cards are revealed simultaneously to avoid anchoring bias.
- If estimates differ significantly, the highest and lowest estimators explain their reasoning.
- The team re-votes until a consensus (or close agreement) is reached, and the value is logged against the story.

6.2 Fibonacci Scale Reference

Points	Relative Meaning
1	Trivial change; well understood, no unknowns (e.g., a config toggle).
2	Simple task with minor logic; low risk.
3	Small feature; a few known moving parts, minor integration.
5	Moderate complexity; some unknowns or cross-component integration.
8	High complexity; significant unknowns, new technology, or multiple integrations.
13	Very complex / large; usually a signal the story should be split further.

6.3 Sprint 1 Estimates & Justification

ID	Points (Fibonacci)	Estimation Justification
US-01	8	High complexity: data pipeline, cleaning, and baseline ML model training involve significant uncertainty and cross-functional effort.
US-02	5	Moderate complexity: requires third-party API integration and model re-training, but builds on the existing pipeline from US-01.
US-06	8	High complexity: optimization algorithm plus a new comparison UI, with tight dependency on forecast accuracy.
US-07	3	Low-moderate complexity: primarily UI/workflow work on top of an already-defined data model.
US-12	8	High complexity: real-time GPS integration, fast-refresh API, and mobile UI work across two platforms (iOS/Android).
US-13	5	Moderate complexity: notification service and preference management are well-understood patterns but need reliable delivery guarantees.

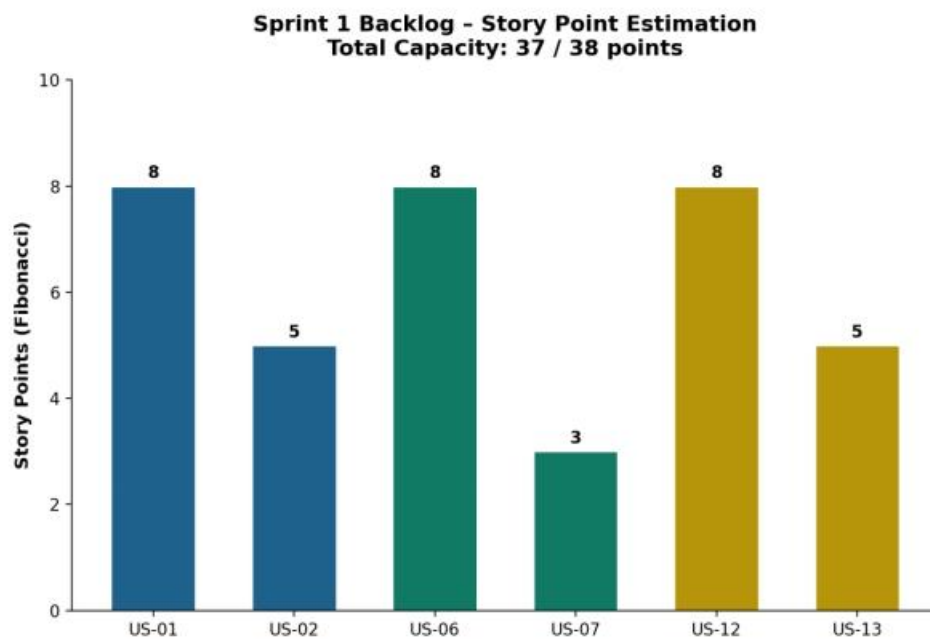


Figure 2: Story point distribution for Sprint 1, grouped by epic.

7. SPRINT GOAL AND EXPECTED DELIVERABLES

7.1 Sprint Goal

By the end of Sprint 1, deliver a working MVP slice of the platform that ingests historical ridership and weather data to forecast demand, generates optimized schedule recommendations for fleet-manager review, and provides commuters with real-time arrival predictions and delay notifications — demonstrating the core value loop of the AI-based demand forecasting system.

7.2 Sprint Details

Sprint Duration	10 working days (2-week sprint)
Sprint Capacity	38 story points (5-member cross-functional team)
Sprint Commitment	37 story points (6 user stories, Epics A, B & D)
Team Composition	1 Scrum Master, 1 Product Owner, 2 Developers, 1 Data Scientist, 1 QA Engineer

7.3 Expected Sprint Deliverables

- A functioning demand-forecasting pipeline trained on historical ridership and live weather data.
- A schedule-optimization module that outputs recommended timetables within an 85% capacity threshold.
- A fleet-manager review interface for approving or editing AI-generated schedule changes.
- A mobile app screen showing real-time, auto-refreshing arrival predictions.
- A push-notification service alerting commuters to delays or schedule changes.
- Automated unit and integration tests covering the forecasting pipeline and notification service.
- A sprint review demo and updated product backlog for Sprint 2 planning.

7.4 Sprint 1 Burndown Plan

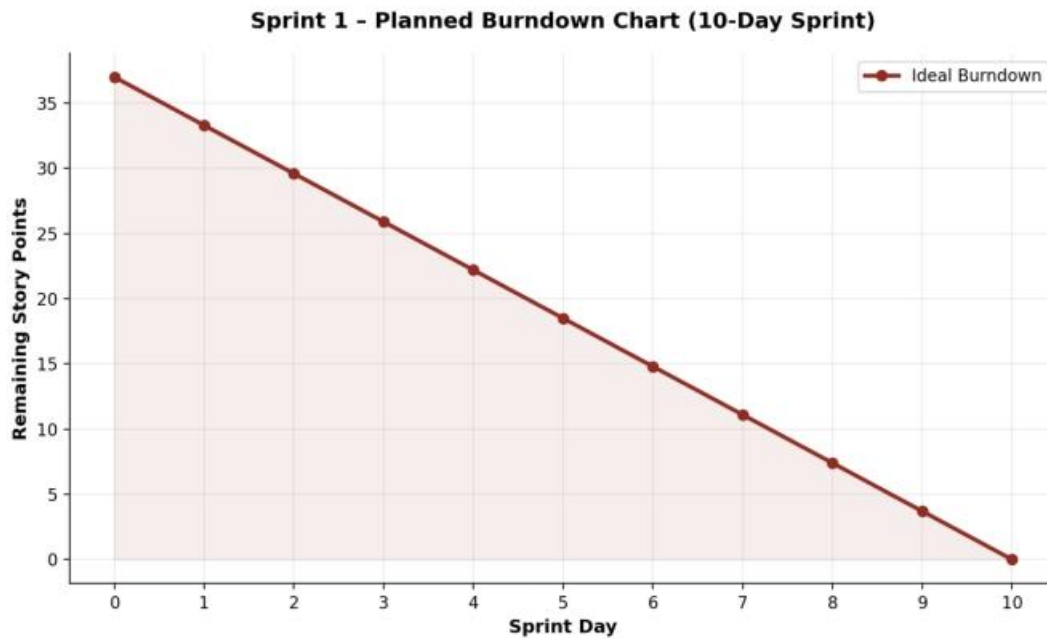


Figure 3: Ideal burndown for the 37-point Sprint 1 commitment.

7.5 Definition of Ready (entry criteria for Sprint Planning)

- Story is written in “As a / I want / so that” format and is small enough to fit within a sprint.
- Acceptance criteria are documented and agreed with the Product Owner.
- Dependencies (see Section 4.2) are identified and, where blocking, resolved or sequenced.
- Story has been estimated by the team using Planning Poker.
- UI/UX mock-ups or API contracts exist where relevant to the story.

7.6 Definition of Done

- Code is peer-reviewed, merged, and passes CI/CD pipeline checks.
- Unit and integration tests pass with $\geq 80\%$ coverage on new code.
- Acceptance criteria for the story are verified by QA.
- Feature is demonstrated in the Sprint Review to the Product Owner and stakeholders.
- Relevant documentation (API/user-facing) is updated.

7.7 Sprint 1 Risk Register

Risk	Likelihood	Impact	Mitigation
Historical ridership data is incomplete or inconsistent	Medium	High	Data profiling in Day 1-2; apply agreed imputation rules; flag gaps to Product Owner early.
Third-party weather API has rate limits/outages	Low	Medium	Cache recent responses; implement graceful fallback to last-known forecast.
GPS feed for US-12 not ready (dependency on US-18)	Medium	Medium	Use a mocked/simulated GPS feed for Sprint 1; integrate real feed in Sprint 2.
Model accuracy below the 85% target	Medium	High	Time-box model tuning; if unmet, demo with documented accuracy and iterate next sprint.
Scope creep from stakeholder requests mid-sprint	Medium	Medium	New requests are logged to the backlog for prioritization, not pulled into the active sprint.

7. CONCLUSION

This case study has analyzed the challenge of mismatched public transportation supply and demand, and proposed an Intelligent AI-Based Smart Public Transportation Demand Forecasting System as a comprehensive solution. By combining AI/ML forecasting, IoT and GPS data integration, and automated schedule and fleet optimization, the proposed platform shifts transit agencies from reactive, manual planning to proactive, data-driven operations. The solution has been designed around core software engineering principles — modularity, scalability, maintainability, reliability, usability, and security — and is best delivered through an Agile Scrum methodology that accommodates the iterative nature of both software development and machine learning experimentation. While risks such as data quality, integration complexity, and adoption resistance must be carefully managed, the expected benefits — improved reliability, reduced waiting times, better fleet utilization, and lower operational costs — make a strong case for pursuing this system as a meaningful step toward sustainable, commuter-centered urban mobility.

References

- Sommerville, I. Software Engineering (10th Edition). Pearson Education.
- Pressman, R. S., & Maxim, B. R. Software Engineering: A Practitioner's Approach. McGraw-Hill Education.
- Schwaber, K., & Sutherland, J. The Scrum Guide. Scrum.org.
- Agile Alliance. Agile Manifesto and Principles. agilealliance.org.